

A software vulnerability detection method based on multi-modality with unified processing[☆]

Wenjing Cai^{a,c}, Junlin Chen^a, Jiaping Yu^a, Wei Hu^a, Lipeng Gao^{b,c},^{*}

^a School of Cybersecurity, Northwestern Polytechnical University, Xi'an 710072, China

^b School of software, Northwestern Polytechnical University, Xi'an 710072, China

^c Research & Development Institute of Northwestern Polytechnical University in Shenzhen, Shenzhen 518063, China

ARTICLE INFO

Keywords:

Vulnerability detection
Code representation
Multimodality
Transformer

ABSTRACT

With the development of the Internet and the Internet of Things, software has become an indispensable part, making software vulnerabilities one of the main threats to computer security. In recent years, a multitude of deep learning-based software vulnerability detection methods have been proposed, especially those based on multimodal approaches. Although these multimodal methods have proven to be effective, they often treat each modality separately. We propose a novel multimodal deep learning method for software vulnerability detection that achieves unified processing of various modalities. This method uses complex network analysis to convert the Code Property Graph into an image-like matrix, obtains key fragments from the source code using code slicing, and then uses a Transformer for function-level vulnerability detection. This enables deeper integration of information from multiple modalities, enhancing detection accuracy. Additionally, it significantly simplifies the model architecture. The result shows that compared to the state-of-the-art methods, our method has improved accuracy by 3%. Furthermore, our approach is capable of detecting some of the vulnerabilities recently released by CVE.

1. Introduction

With the advancement of technology, the number of software applications is increasing, and their size is also growing. Consequently, developers are bound to write code with vulnerabilities during the coding process. If these vulnerabilities are related to security, they may be exploited by hackers, resulting in immeasurable losses [1]. This has sparked widespread research among scholars, especially deep learning-based detection methods [2,3].

Many deep learning-based methods have been proposed to detect software vulnerabilities. Computers can only process input in numerical form, so the first step of all vulnerability detection methods is to represent the code as vectors. There are currently three mainstream methods for vector representation: treating code as text, representing code as a graph, and multimodal representation [4–7]. Consequently, vulnerability detection methods can be divided into three major categories.

The method of treating code as text often employs Recurrent Neural Networks (RNNs) [8] to detect vulnerabilities, with this category of

methods typically using Bidirectional Long Short Term Memory (BiLSTM) [9] as the backbone network [10]. Meanwhile, with the success of the transformer, many scholars are also exploring the application of transformers in software vulnerability detection [11]. Their common practice is to slice the source code to address the issue of long computational times with RNNs. Then, according to the characteristics of the programming language, tokenization rules are established to split the source code into tokens. Finally, after converting them into vectors, they can be input into the network.

After representing code as a graph, Graph Neural Networks (GNNs) [12] are typically used to complete the detection task. The Gated Graph Neural Network (GGNN) [13] is the mainstream backbone network in this category, because the Gated Recurrent Unit (GRU) [14] allows it to better capture dependency information. Graphs can better model the control and data dependencies between code statements, and also better express the hierarchical relationships between codes to enable detection at various granularities. These methods [15,16] generally involve the use of tools to extract various graph representations from the

[☆] This work was supported in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2023A151511175, in part by the National Natural Science Foundation of China under Grant 42101469, and in part by the Aeronautical Science Foundation of China under Grant 2023Z034053001.

^{*} Corresponding author at: School of software, Northwestern Polytechnical University, Xi'an 710072, China.

E-mail address: gaolipeng@nwpu.edu.cn (L. Gao).

source code, such as the Code Property Graph (CPG) [17] and the Abstract Syntax Tree (AST) [18]. Subsequently, the graphs are represented as vectors, and finally they are input into GNNs for detection.

With the continuous improvement of computational performance, it has become easier to process multiple modalities simultaneously, leading to an increase in the number of multimodal-based methods. These methods can integrate multiple representations of code, such as source code, assembly code, and graphical forms [19–21]. This allows the model to access more information, which often leads to better performance. For instance, Tao proposed an accurate method by enhancing and fusing the features of source code and assembly code. The main innovation is using a cross-slice method to generate aligned dual-modality program slices of source code and assembly code, that is, first slicing the source code and obtaining the corresponding assembly code, and then slicing the assembly code to obtain the corresponding source code [22].

Although these methods have achieved good results, there are still areas for improvement. Methods that treat code as text struggle to represent the dependencies between code segments, which can lead to a higher rate of false positives. RNNs also have computational dependencies, and to control computation time, it is often necessary to perform code slicing. Although graphs can represent structural information well, GNNs typically focus on the local neighborhood information of nodes and struggle to capture long-range dependencies or global graph structural information. Limited by the issue of over-smoothing, GNNs cannot be made deeper, which further restricts the accuracy of the methods [23]. Multimodal methods use various forms to represent code, but they require different models to process different modalities. For instance, RNNs or Transformer are used to handle text, GNNs are utilized for graph representations, and CNNs along with their variations are employed to process images. These methods have not achieved unified processing of modalities, resulting in more complex models, and the integration of modalities can only perform simple concatenation of the extracted features, which limits the effectiveness of the fusion.

The inputs for multimodal models are typically image and text. Our previous work involved converting text into a matrix format similar to image [24]. Meanwhile, Transformer is extensively used in the field of multimodality [25]. This inspires us that integrating our previous work with Transformer could enable unified modality processing in software vulnerability detection. Therefore, we proposed a novel multimodal method for software vulnerability detection. First, the source code is preprocessed and represented as CPG. Building on our previous work, we use complex social network analysis to convert CPG into a matrix similar to an image, serving as the image input. Then, based on the characteristics of the programming language, we establish tokenization rules to divide the code into tokens, which serve as the text input. Finally, using the method of Vision-Language Transformer (ViLT), both the image and text are represented as vectors, with the addition of [CLS] token, positional encoding, and type encoding. The [CLS] token is used as the standard for detection.

In summary, this paper makes the following contributions:

- (1) A new function-level software vulnerability detection method has been proposed. This method leverages social network analysis to transform the CPG into an image-like matrix as the graphical input, while slicing the code to serve as the textual input. Subsequently, a Transformer model is employed for vulnerability detection, achieving unified processing across multiple modalities. This enables the model to integrate information from multiple modalities more deeply while significantly simplifying the model architecture.
- (2) Our method has demonstrated significant improvements compared to SvdGP. We achieved a higher accuracy rate of approximately 3%.
- (3) In the latest vulnerabilities reported by CVE, some real vulnerabilities were detected during testing.

The remainder of the paper is organized as follows: Related works, Method, Experiment, Discussion, Conclusion.

2. Related works

We classify existing deep learning-based software vulnerability detection methods into three major categories based on the representation of code: treating code as text, representing code as graphs, and multimodal representation.

2.1. Methods that treat code as text

Hanif introduced a deep learning method called VulBERTa for detecting security vulnerabilities in source code [26]. VulBERTa involves pretraining a RoBERTa model to learn deep knowledge representations of the syntax and semantics of code, which are then utilized to train a vulnerability detection classifier. A customized tokenization process is employed to handle code from open-source C/C++ projects, combining the Byte Pair Encoding algorithm with predefined code tokens, to better represent the code while maintaining its original syntactic structure. Fu proposed a Transformer-based vulnerability detection method called LineVul [27]. LineVul employs a pretrained language model, CodeBERT, to generate vector representations of the source code. Utilizing the BERT architecture, which captures long-term dependencies within the code through layers of self-attention mechanisms. It uses the output of the BERT model for binary classification to determine whether a function is vulnerable. For functions predicted to be vulnerable, LineVul leverages the internal self-attention mechanism of the BERT model to pinpoint the specific lines of code where the vulnerabilities are located.

2.2. Methods that represent code as graphs

Meng proposed a novel vulnerability detection method called HybridNN [28]. They first simplified the CPG and designed a Hybrid Code Graph (HCG) that is more suitable for deep semantic extraction through GNNs. Subsequently, they utilized a GNN model with a hierarchical attention mechanism to extract deep semantics from heterogeneous graphs and applied it to the newly designed HCG representation. In addition, the paper also introduces a UD-Sampling method, which combines up-sampling and down-sampling techniques to balance the distribution of training samples. Wen introduced a novel vulnerability detection framework called AMPLE [29], which mainly consists of two parts: graph simplification and enhanced graph representation learning. Graph simplification aims to simplify the structure of the graph by reducing the distance between nodes through the minimization of the size of nodes in the code structure graph. Enhanced graph representation learning includes two modules: one is the Edge-Aware GNN module, which integrates heterogeneous edge information into the node representation; the other is the Kernel-Scale Representation module, designed to capture relationships between distant nodes in the graph.

2.3. Methods that use multimodal representation

Ni proposed a method with three modality [20]. The text is the sequence of source code; the graph is represented using the CPG; a visualization tool is used to convert CPG to images. In terms of the model, four encoders were used. The Image Encoder uses SwinV2 [30] to model the image. The Code Encoder employs a pre-trained model for the programming domain to obtain the vector representation of the code. The Positional Encoder uses OCR technology to obtain the position of each symbol in the image. The Multi-modal Graph Encoder is divided into two stages: first, using GAT to learn the CPG, then integrating the results of the Positional Encoder into the learned results, and finally using GCN to detect. Guo proposed a method that integrates graph-level and token-level information [21]. The graph-level feature extraction module represents the source code as CPG. Then, a GGNN is used to extract features. The token-level feature extraction module

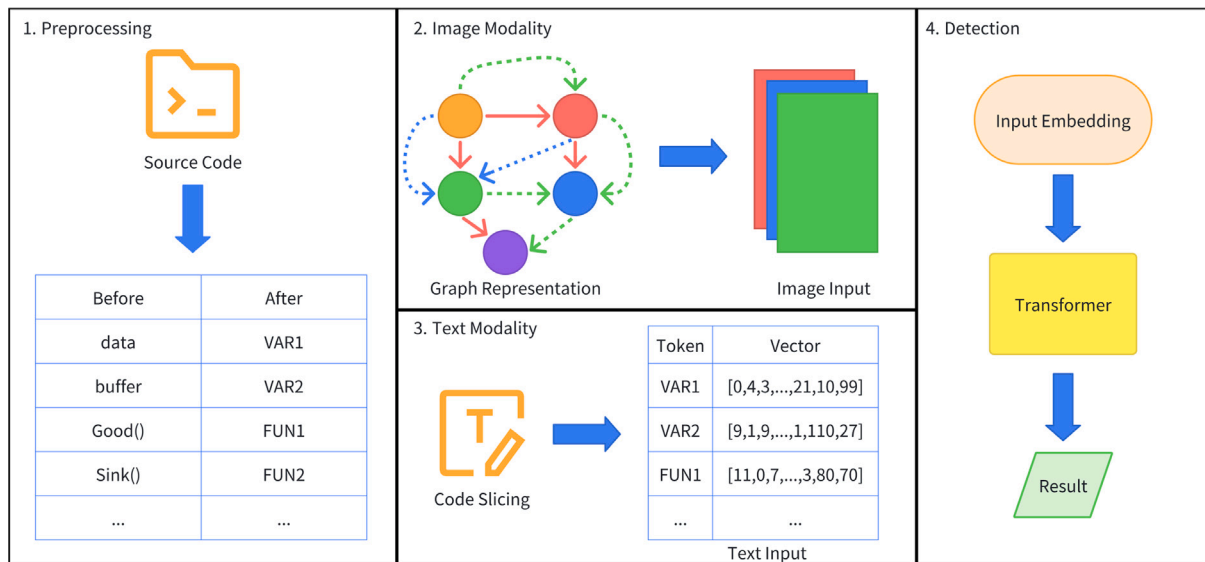


Fig. 1. Method overview.

obtains the taint propagation chain in the source code through taint analysis, and then uses BiLSTM with a self-attention mechanism to extract features. Finally, the classification module is used to integrate the two types of features and uses a pre-trained Xgboost model for binary classification.

3. Method

This section provides a detailed description of the method, which can be divided into three main parts, as depicted in Fig. 1. First, the code preprocessing stage standardizes custom variables and functions while removing comments. Second, inputs are generated for both textual and graphical aspects. The code is sliced and tokenized to create the textual input, while it is transformed into a graph and then converted into an image using centrality measures. Third, the model’s details are presented, providing a clear overview of the entire process from input to output.

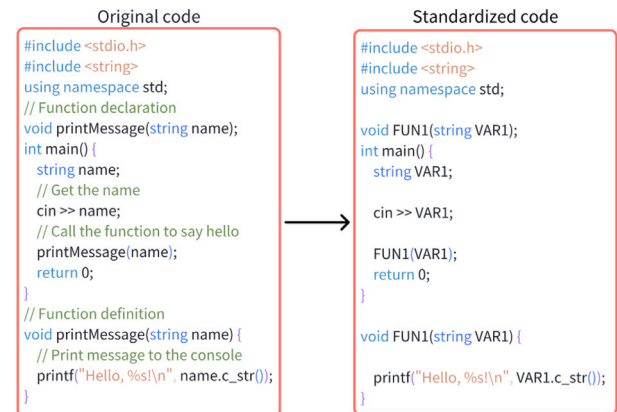


Fig. 2. Example of code preprocessing.

3.1. Code preprocessing

Comments are often added to improve code readability. However, since they are not crucial to the code's semantics, they can be safely omitted. Additionally, standardizing custom variable and function names reduces coding complexity and ensures consistency across code segments with identical semantics, thereby enhancing detection accuracy.

Engineering projects often use third-party library functions, which involve variables or keywords, collectively known as library reserved words. Library functions, such as “fork()” for creating child processes on Unix systems, library variables like the global variable “errno” for recording error messages in Unix, and library reserved words such as “WINAPI” and “CALLBACK” used as modifiers in Windows, carry the same semantic significance as reserved words in programming languages. Therefore, they should be preserved as well. An example of code preprocessing is depicted in Fig. 2.

We use regular expressions to extract variable names and function names from the code while maintaining a reserved keyword set that includes C/C++ language keywords and those from third-party libraries, as shown in Fig. 3. Any variable or function name identified by the regular expressions that exists in the reserved keyword set will not be standardized. This ensures that critical semantic information associated with these names is preserved.

Reserved words covered by the C++17 standard	'override', 'private', 'constexpr', 'reinterpret_cast', 'static_assert', 'template', 'optional', 'thread_local'
Reserved words used in Linux	'EXPORT_SYMBOL', 'MODULE_LICENSE', '__init', 'kmalloc', 'spin_lock', 'wait_event', 'pr_info'
Reserved words used in FFmpeg	'AVCodec', 'AVFrame', 'AVPacket', 'avcodec_find_decoder', 'av_read_frame', 'swr_alloc',
Reserved words used in QEMU	'CPUState', 'QEMUFile', 'MemoryRegion', 'qemu_system_reset', 'ioport_write', 'cpu_exec'
Regular expression for identifying comments	(?<!)\{\}\ \.\ *\ \\\$ \\})*\{*\}
Regular expression for identifying variables	\b([_A-Za-z]\w*)\b(?:=\s*(\
Regular expression for identifying functions	\b([_A-Za-z]\w*)\b(?:?:=\s*(\w+)(\((?!\s*\w+))?(?\s*\()

Fig. 3. Partial reserved words and the regular expressions used.

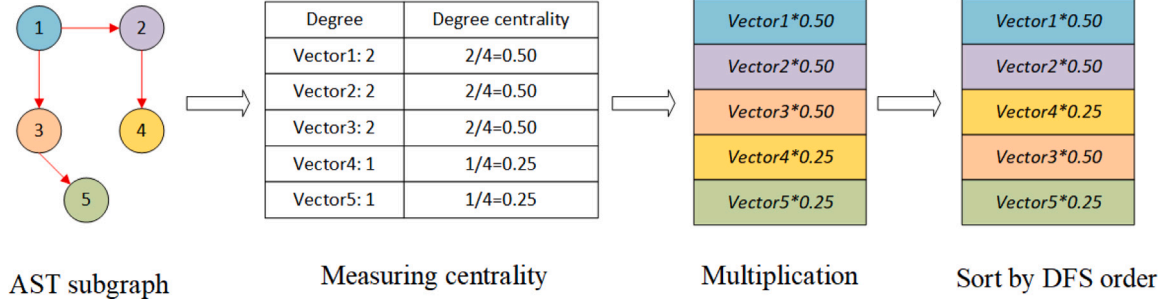


Fig. 4. Example of AST subgraph image generation on degree centrality.

Different third-party libraries may have different reserved keywords, but these keywords can also be extracted using these regular expressions and added to the reserved keyword set to ensure their preservation during the standardization process. This entire process can be automated through a program, eliminating the need for manual intervention, making it both cost-effective and scalable.

3.2. Intermediate representation of code

The second step is to generate the intermediate representation of the code, where this paper adopts both image and text as the inputs for the model. This section will provide a detailed introduction on how to generate image and text.

3.2.1. Image input

We use centrality measures to convert the graph into an image. Centrality is a commonly used concept in complex network analysis, which expresses a vertex's importance in the whole network. Nodes in the graph represent specific semantic information, while the edges between them provide structured information. Centrality measures evaluate the importance of each node by considering its individual characteristics (semantic information) and its relationships with other nodes (structured information), helping to quantify the significance of nodes in the graph. This is similar to the attention mechanism in neural networks. Applying such importance as weights to the node vectors can highlight more critical information. Our previous work demonstrated that this matrix-based representation does not result in a loss of semantic information compared to the original graph representation [24].

In our method, three kind of centrality, namely degree centrality, closeness centrality, and eigenvector centrality, are utilized.

degree centrality:

$$x_i = \frac{\deg(i)}{N-1} \quad (1)$$

Note that $\deg(i)$ denotes the degree of the i th node, and N represents the number of nodes present in the graph. This means that the node's significance increases with its extensive connections and larger number of neighboring nodes.

closeness centrality:

$$x_i = \frac{N-1}{\sum_{j \neq i} d_{ij}} \quad (2)$$

Note that d_{ij} denotes the shortest direct distance between node i and node j . If a node has a short distance from every point in the network, then it is not subordinate to other nodes.

eigenvector centrality:

$$x_i = \alpha \sum_j A_{ij} x_j + \beta \quad (3)$$

Note that A refers to the adjacency matrix of the graph, and its Eigenvalue is represented by λ . The initial value of centrality is denoted by β , while α must satisfy the condition $\alpha < \frac{1}{\lambda_{\max}}$. The significance of a

node is directly proportional to the importance of the nodes it connects to.

Subgraph partition refers to partitioning the CPG according to edge type [31]. Since there are three distinct types of edges in the CPG, the influence of different types of edges on nodes can be better obtained after partition. This involves partitioning the CPG into subgraphs based on the edge type, resulting in AST, PDG and CFG [32,33].

The process of image generation is illustrated clearly in Fig. 4, which includes measuring centrality, multiplication and sort by the depth-first search (DFS) order. The centrality measures refer to calculating the centrality of each node according to the corresponding formula. After measuring the centrality of each node, we multiply the vector representation of each node by its corresponding centrality to form a preliminary image. Finally, We rearrange the multiplied node features in the order of DFS.

3.2.2. Text input

The processing of text is relatively straightforward. We first perform code slicing based on the API to extract the code that may have vulnerabilities, as shown in Algorithm 1. Then, we establish tokenization rules according to the syntax of C/C++ to divide these codes into tokens. These can then be used as the textual input for the model, and Word2Vec is used to obtain the vector representation for each token [34].

Algorithm 1 GenerateCodeSlice

```

1: Initialize an empty set Slice to store the slicing result
2: Initialize an empty set Visited to keep track of visited nodes
3: function GENERATESLICE(SystemCall, EntryPoint)
4:   if EntryPoint corresponds to SystemCall then
5:     Add EntryPoint to Slice
6:     Mark EntryPoint as visited
7:   end if
8:   for all statements s in EntryPoint do
9:     if s is a control-flow statement and not visited then
10:      Add all reachable statements from s to Slice
11:      Mark all reachable statements as visited
12:    end if
13:  end for
14:  for all statements s in Slice do
15:    if s calls another function f and f is not visited then
16:      Recursively call GenerateSlice(f, SystemCall)
17:    end if
18:  end for
19: end function

```

3.3. Classification

By using the intermediate representation method of the code mentioned above, we can use the transformer model to uniformly process

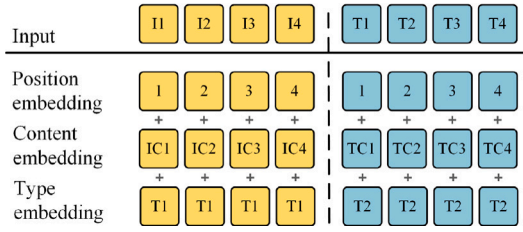


Fig. 5. Diagram of the embedding representation.

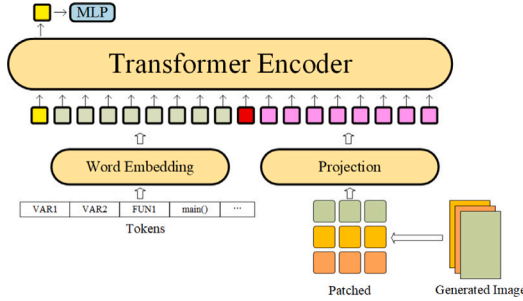


Fig. 6. Model architecture.

multiple modalities. There is no need to adopt different models for each modality as other methods do. In our method, both image and text are processed into individual patches. After being embedded, they can be input into the transformer. The treatment of images is similar to Vision Transformer (ViT) [35], where the image is divided into multiple 16×16 patches, which are then mapped into 768-dimensional vectors. For text, it is only necessary to represent each token as a 768-dimensional vector. The self-attention mechanism captures global information, and each patch, whether it is an image or text, has some semantic information associated with its position, so positional encoding is added here to represent it [36]. To distinguish between different modalities, type information is also included. Finally, to accomplish the vulnerability detection task, we add a '[CLS]' token at the head of the sequence as a basis for classification. The specific process of the embedding representation is shown in Fig. 5.

The architecture of our model is shown in Fig. 6. The goal of feature extraction is to determine whether the code contains vulnerabilities or not. Therefore, a fully connected layer is used to transform the feature dimensions of the '[CLS]' token into two, corresponding to the presence or absence of vulnerabilities. Finally, a softmax layer is applied to normalize these two values so that their sum equals 1, and the normalized values can be regarded as the respective probabilities of being vulnerable or not [37]. During training, a cross-entropy loss function is used to measure the classification effectiveness and to optimize the parameters [38–40].

4. Experiment

Table 1 provides a detailed overview of the model's parameters, which were fine-tuned through multiple training iterations. Notably, to prevent overfitting, we set the dropout rate to 0.3 and used five-fold cross-validation for both training and testing. The training set was proportioned at four times the size of the validation set to ensure robust evaluation. We begin this section by presenting four research questions, then provide a brief overview of the dataset employed, and conclude with an analysis of the experimental results.

Table 1
Parameter settings.

Parameter	Setting
precision	16
loss function	Cross Entropy
activation function	ReLU
optimizer	Adam
batch size	32
learning rate	0.0001
dropout	0.3
epoch num	100
vit model	vit_base_patch16_224
image_size	224
patch_size	16
num_heads	8
num_layers	4

4.1. Research question

To clearly demonstrate the effectiveness of our method and the necessity of certain steps, we proposed four research questions and provided answers through experimental results.

RQ1: How well does our proposed method perform? Specifically, we aimed to evaluate how well our method performs in comparison to other SOTA methods.

RQ2: How much performance improvement does multimodal fusion bring? We separately assess the performance of each individual modality and then compare it with the result of multimodal fusion.

RQ3: Is it necessary to have code slices on the text side? How much performance improvement does it bring?

RQ4: How does our method perform in the latest vulnerabilities reported by CVE?

4.2. Dataset

We conducted experiments on two datasets whose Github links are <https://github.com/CGCL-codes/VulCNN> and <https://github.com/jumormt/DeepWukong>, called A and B respectively. These data are usually collected from two open-source vulnerability libraries, namely, SARD (Software Assurance Reference Data Set) and CVE (Common Vulnerabilities and Exposures). The specific types and corresponding numbers of vulnerabilities can be found in Table 2.

4.3. Selected baselines

We selected two traditional vulnerability detection tools and six deep learning-based detection methods for comparison. RATS scans source code for potential security vulnerabilities in C, C++, Perl, Python, and PHP, focusing on identifying insecure functions and patterns. Flawfinder is a static code analysis tool for C/C++ that searches for potential security flaws by matching source code against a database of known insecure coding practices, ranking them by severity.

Vulcnn [41] utilizes centrality measures to transform PDG into image-like matrices and applies a TextCNN model for detection. VulDeeP-ecker [10] detects vulnerabilities by extracting "code gadgets" via heuristic slicing, vectorizing them with Word2Vec, and using a BiLSTM model to capture contextual dependencies. Devign [15] uses GGNN to analyze CPG, capturing both semantic and structural code dependencies. By leveraging message-passing and comprehensive program representations, it effectively detects vulnerabilities in real-world datasets. VGDetect [4] combines GNNs with textual embeddings to capture both structural and semantic information for accurate detection. SvdGP [24] utilizes a sophisticated CPG representation of code, partitions the CPG into subgraphs based on edge types, arranges node vectors in depth-first search order, and employs TextCNN for vulnerability detection. HyVulDect [19] is a hybrid vulnerability detection system combining graph and text. It uses CPG for structural and

Table 2
The distribution of vulnerability types.

Dataset-ID	CWE-ID	Description	Number of samples
A	NULL	non-vulnerability samples	26,970
A	CWE-78	OS Command Injection	963
A	CWE-121	Stack Based Buffer Overflow	3,558
A	CWE-122	Heap Based Buffer Overflow	2,122
A	CWE-124	Buffer Underwrite	1,129
A	CWE-126	Buffer Over-read	704
A	CWE-127	Buffer Under-read	1,014
A	CWE-134	Uncontrolled Format String	2,208
A	CWE-195	Signed to Unsigned Conversion Error	499
B	NULL	non-vulnerability samples	774,714
B	CWE-20	Improper Input Validation	19,617
B	CWE-22	Improper Limitation of a Pathname to a Restricted Directory	2,764
B	CWE-78	OS Command Injection	1,729
B	CWE-119	Improper Restriction of Operations within Memory Buffer	18,928
B	CWE-125	Out-of-bounds Read	3,645
B	CWE-190	Integer Overflow or Wraparound	1,052
B	CWE-254	Security Features	27,776
B	CWE-399	Resource Management Errors	12,367
B	CWE-400	Uncontrolled Resource Consumption	1,872
B	CWE-787	Out-of-bounds Write	12,946

semantic code representation and extracts features with GGNN and BiLSTM models. The system fuses these features and classifies them using an XGBoost classifier, enabling accurate and robust vulnerability detection.

4.4. Evaluation metrics

In our evaluation, we use False Positive Rate (FPR), False Negative Rate (FNR), and Accuracy Rate (ACC) as the evaluation metrics. We define T as correct predictions, F as wrong predictions, P as Vulnerable, and N as Non-Vulnerable.

FPR stands for False Positive Rate, which is a statistical metric used to evaluate the performance of binary classification models. It represents the proportion of negative instances that are wrongly classified as positive by the model.

$$FPR = \frac{FP}{FP + TN} \quad (4)$$

FNR stands for False Negative Rate, which is a statistical metric used to evaluate the performance of binary classification models. It represents the proportion of positive instances that are wrongly classified as negative by the model.

$$FNR = \frac{FN}{FN + TP} \quad (5)$$

ACC stands for Accuracy, which is a statistical metric used to evaluate the performance of classification models. It represents the proportion of correctly classified instances out of the total number of instances.

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

We conducted a one-tailed significance test with the null hypothesis that the performance of the proposed method is less than or equal to that of the baseline model. The test method employed was a paired t -test, with the confidence level set at 0.05. Each method was independently run 10 times, and 10 accuracy observations were recorded. Subsequently, the proposed method was compared with each baseline model using paired t -tests to evaluate whether the performance differences were statistically significant.

First, calculate the mean of the differences:

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n (x_i - y_i) \quad (7)$$

Next, calculate the standard deviation of the differences:

$$S_d = \sqrt{\frac{\sum_{i=1}^n (d_i - \bar{d})^2}{n - 1}} \quad (8)$$

Subsequently, the standard error can be obtained:

$$SE = \frac{S_d}{\sqrt{n}} \quad (9)$$

This allows the t -statistic to be calculated:

$$t = \frac{\bar{d}}{SE} = \frac{\bar{d}}{S_d / \sqrt{n}} \quad (10)$$

The degree of freedom in a t -test is $n - 1$. Based on the t -statistic and the degree of freedom, the p -value can be determined:

$$P = P(T \geq t \mid df) \quad (11)$$

if $P \leq 0.05$, the null hypothesis is rejected, indicating that the performance of the proposed method is significantly better than the baseline model.

We also calculated the Cohen's d effect size for accuracy:

$$d = \frac{\bar{y} - \bar{x}}{s} \quad (12)$$

Where s represents the pooled standard deviation, which is calculated as:

$$s = \sqrt{\frac{(n-1)s_{d_1}^2 + (n-1)s_{d_2}^2}{2n-2}} \quad (13)$$

4.5. Result and analysis

Answer to RQ1

To evaluate the effectiveness of our proposed method, we compared it with several other vulnerability detection tools. Specifically, we compared it with two static analysis tools (FlawFinder and RATS) and six deep learning-based vulnerability detection methods (VulCNN, VulDeePecker, Devign, VGDetect, SvdGP, and HyVulDect).

As depicted in Table 3, the FPR and FNR of the two open-source static analysis tools are less than optimal. This can be attributed to their reliance on simplistic parsing methods and human-defined vulnerability rules. Despite the use of handcrafted rules to identify vulnerabilities, it is inherently difficult to cover the full spectrum of scenarios and edge cases. As a result, the tools' capacity for precise vulnerability detection is compromised, resulting in elevated rates of both false positives and false negatives.

Our method has several advantages over other deep learning approaches. VulCNN [41] is the first approach to represent code in an image-like manner using centrality measures, but it employs a relatively simple TextCNN model, which has notably inferior learning capabilities compared to the Transformer [42]. While VulDeePecker [10]

Table 3Comparison with other methods on two datasets, where the p -value and effect size are calculated based on accuracy.

Model/Metrics	dataset A					dataset B				
	FPR(%)	FNR(%)	ACC(%)	pV	ES	FPR(%)	FNR(%)	ACC(%)	pV	ES
our	11.02	15.73	92.38	–	–	11.31	15.34	93.11	–	–
RATS	27.91	64.43	59.17	0.000	25.506	29.11	59.35	61.42	0.000	24.808
FlawFinder	54.67	47.62	59.08	0.000	48.682	55.81	45.68	59.73	0.000	35.144
VulCNN	20.82	9.82	83.19	0.000	14.089	17.26	14.61	84.37	0.000	14.240
VulDeePecker	25.37	14.95	76.68	0.000	16.610	32.49	28.94	70.26	0.000	11.948
Devign	24.76	13.37	83.06	0.000	12.864	22.31	14.76	82.69	0.000	12.015
VGDetect	26.14	16.62	78.53	0.000	10.896	24.83	14.27	80.31	0.000	18.653
SvdGP	11.39	13.89	89.27	0.002	1.385	12.54	13.11	91.74	0.002	1.322
HyVulDect	11.44	16.36	90.41	0.000	1.685	11.92	16.11	92.20	0.002	1.387

Table 4

Comparison of modality.

Task/Metrics	FPR(%)	FNR(%)	ACC(%)
Multi-modality on A	11.02	15.73	92.38
Image-modality on A	16.40	19.80	87.54
Text-modality on A	22.54	29.11	78.74
Multi-modality on B	11.31	15.34	93.11
Image-modality on B	15.87	18.74	88.35
Text-modality on B	20.71	26.80	81.41

utilizes system APIs to extract code gadgets, its method ignores crucial dependency information in the code, resulting in high FPR and FNR rates. Similarly, although Devign [15] employs CPG to represent code and uses GGNN for vulnerability detection, its reliance on message passing and numerous edges in CPG can make it computationally expensive. Since VGDetect [4] only considers the control dependency between codes, and the data dependency is not expressed, so its performance is less competitive. Inspired by VulCNN, SVDGP [24] uses CPG to represent code and employs subgraph partitioning to better capture features. It also uses a relatively simple TextCNN model to demonstrate that the improvement in performance comes from the representation of the code. HyVulDect [19] is also a multimodal approach, utilizing GNN to process the graph representation of code and BiLSTM to handle the token representation of code. Finally, it simply concatenates the features obtained from GNN and BiLSTM as the final features. Compared to our method, its capability in modality fusion is inferior, hence its performance is worse than our method.

Answer to RQ2

We conducted an exploration to understand the impact of integrating multiple modalities on the performance of our model, as depicted in Table 4. To this end, we conducted a comparative analysis across two distinct datasets, where we assessed the efficacy of multimodal data, image modality alone, and text modality alone. The findings from our study clearly demonstrate that the multimodal fusion technique has successfully harnessed the unique attributes of both images and text, resulting in a marked enhancement in performance.

In particular, the image modality has shown a substantial improvement over the text modality, primarily due to its ability to incorporate structured information inherent in the code. This integration provides a richer and more comprehensive representation of the data, allowing the model to capture complex patterns and relationships that might be overlooked by a text-only approach.

Moreover, the multimodal fusion not only leverages the strengths of individual modalities but also compensates for their individual limitations. For instance, while images can convey spatial and structural information, text provides semantic and contextual insights. By combining these two modalities, our model is better equipped to understand the nuances of the data, leading to more accurate and robust predictions.

Answer to RQ3

We performed ablation studies on two datasets to assess the impact of code slicing. The results in Table 5 demonstrate that code slicing

Table 5

Effect of slicing.

Task/Metrics	FPR(%)	FNR(%)	ACC(%)
with slicing on A	11.02	15.73	92.38
without slicing on A	12.31	16.64	90.17
with slicing on B	11.31	15.34	93.11
without slicing on B	13.44	15.27	89.84

has enhanced the accuracy. This improvement is due to the fact that, code slicing allows for a more nuanced and context-aware examination of the source code. By focusing on the specific paths and constructs that are most relevant to the execution of a program, code slicing provides a more detailed and targeted view of the code's structure. This, in turn, enables the model to better understand the flow and logic of the program, which is crucial for accurately identifying potential vulnerabilities.

Answer to RQ4

To verify whether our method can detect the latest vulnerabilities, we collected some recent vulnerabilities from the CVE database for validation. As shown in Table 6, our method has detected multiple vulnerabilities in the open source projects Linux, FFmpeg, and QEMU.

For example, Fig. 7 shows a vulnerability that involves null pointer dereference in versions of the Linux kernel that are earlier than 5.15.13. The main reason for the vulnerability is that, in the event of an error, the code expects a null pointer to be returned, but instead, the wrong pointer is returned. This is because the “!” operator can only determine whether the pointer is null, whereas the “IS_ERR” macro function can determine whether the pointer is correct. CVE-2023-26545 is a vulnerability about double free which may happen in the Linux kernel before 6.1.13. when MPLS fails to register the sysctl table under new location the old pointers will not get overwritten and may be freed again (see Fig. 8).

5. Discussion

Threats to validity. The labeled functions in SARD are from synthetic programs, which may not be representative of entire programs. To address this issue, we added some real-world vulnerabilities from CVE and non-vulnerable functions from open-source products to our dataset. Furthermore, incorrect labeling of the data may also impact the validity of the results.

Model selection. The backbone of our model is the vit_base_patch 16_224 version of the ViT model. There may be variations in accuracy across different versions of the ViT model. Being a smaller model, it has a smaller number of parameters and requires fewer computational resources. However, the generalization capability of a smaller model may be somewhat limited. If there is a need to handle images of different sizes, additional adjustments to the model may be required.

5	6	+ #include <linux/err.h>
5	6	#include "dr_types.h"
6	7	
7	8	#define DR_DOMAIN_SW_STEERING_SUPPORTED(dmn, dmn_type) \
↓		@@ -72,9 +73,9 @@ static int dr_domain_init_resources(struct mlx5dr_domain *dmn)
72	73	}
73	74	
74	75	dmn->uar = mlx5_get_uars_page(dmn->mdev);
75	-	if (!dmn->uar) {
76	+	if (IS_ERR(dmn->uar)) {
76	77	mlx5dr_err(dmn, "Couldn't allocate UAR\n");
77	-	ret = -ENOMEM;
78	+	ret = PTR_ERR(dmn->uar);
78	79	goto clean_pd;
79	80	}

Fig. 7. CVE-2023-23006 vulnerability and patch source code.

↑		@@ -1428,6 +1428,7 @@ static int mpls_dev_sysctl_register(struct net_device *dev,
1428	1428	free:
1429	1429	kfree(table);
1430	1430	out:
1431	+	mdev->sysctl = NULL;
1431	1432	return -ENOBUFFS;
1432	1433	}
1433	1434	
↑		@@ -1437,6 +1438,9 @@ static void mpls_dev_sysctl_unregister(struct net_device *dev,
1437	1438	struct net *net = dev_net(dev);
1438	1439	struct ctl_table *table;
1439	1440	
1441	+	if (!mdev->sysctl)
1442	+	return;
1443	+	
1440	1444	table = mdev->sysctl->ctl_table_arg;
1441	1445	unregister_net_sysctl_table(mdev->sysctl);
1442	1446	kfree(table);

Fig. 8. CVE-2023-26545 vulnerability and patch source code.

Table 6

A part of latest vulnerabilities detected by our method.

Project	CVE-ID	Vulnerable file	Release date
Linux	2023-28772	lib/seq_buf.c	03/23/2023
Linux	2023-26545	net/mpls/af_mpls.c	02/25/2023
Linux	2023-23006	drivers/net/ethernet/mellanox/mlx5/core/steering/dr_domain.c	01/11/2023
Linux	2023-23003	tools/perf/util/expr.c	01/11/2023
Linux	2022-47519	drivers/net/wireless/microchip/wilc1000/cfg80211.c	12/18/2022
Linux	2022-3910	io_uring/msg_ring.c	11/09/2022
FFmpeg	2022-3109	libavcodec/vp3.c	09/02/2022
FFmpeg	2021-38114	libavcodec/dnxhddc.c	08/04/2021
FFmpeg	2021-33815	libavcodec/exr.c	06/03/2021
QEMU	2022-4172	hw/acpi/erst.c	11/28/2022
QEMU	2022-0358	tools/virtiofsd/passthrough_ll.c	01/25/2022
QEMU	2021-3546	contrib/vhost-user-gpu/virgl.c	05/10/2021

6. Conclusion

In this paper, we propose a novel multimodal deep learning-based method for software vulnerability detection. Our method takes two types of modalities as input, namely the image modality and the text modality. For the image modality, we first obtain the CPG of the code

using Joern, and then convert it into an image-like matrix using centrality measures. The matrix is then divided into 16×16 patches, and each patch is represented as a vector for input. For the text modality, we first use code slicing to obtain key fragments, then tokenize them according to the syntax rules of C/C++, and finally use Word2Vec to obtain the vector representation for each token as input. We then utilize Transformer for feature extraction and vulnerability detection.

The key contribution of this paper is the unified processing of multiple modalities, which enhances detection performance through deeper modality integration and eliminates the need for separate models for each modality. Our proposed method achieves an accuracy of 92.38%, a false positive rate of 11.02%, and a false negative rate of 15.73% on the SARD and CVE datasets. Additionally, our method is capable of detecting some of the most recently published vulnerabilities by CVE. However, there is still room for improvement, and we will continue to explore and enhance the effectiveness of our method in the future.

CRedit authorship contribution statement

Wenjing Cai: Writing – original draft, Methodology, Conceptualization. **Junlin Chen:** Writing – review & editing, Software. **Jiaping Yu:** Visualization, Investigation. **Wei Hu:** Formal analysis, Investigation, Resources. **Lipeng Gao:** Supervision, Methodology, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] H. Zhang, S. Wang, H. Li, T.-H. Chen, A.E. Hassan, A study of C/C++ code weaknesses on stack overflow, *IEEE Trans. Softw. Eng.* 48 (7) (2021) 2359–2375.
- [2] Y. Wan, S. Zhang, H. Zhang, Y. Sui, G. Xu, D. Yao, H. Jin, L. Sun, You see what I want you to see: poisoning vulnerabilities in neural code search, in: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*, pp. 1233–1245.
- [3] Y. Wan, W. Zhao, H. Zhang, Y. Sui, G. Xu, H. Jin, What do they capture? - A structural analysis of pre-trained language models for source code, in: *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022*, pp. 2377–2388.
- [4] X. Cheng, H. Wang, J. Hua, M. Zhang, G. Xu, L. Yi, Y. Sui, Static detection of control-flow-related vulnerabilities using graph embedding, in: *24th International Conference on Engineering of Complex Computer Systems, ICECCS 2019*, pp. 41–50.
- [5] Y. Sui, X. Cheng, G. Zhang, H. Wang, Flow2vec: Value-flow-based precise code embedding, *Proc. ACM Program. Lang.* 4 (2020) 1–27.
- [6] Y. Sui, J. Xue, Value-flow-based demand-driven pointer analysis for C and C++, *IEEE Trans. Softw. Eng.* 46 (8) (2018) 812–835.
- [7] X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, Y. Sui, How about bug-triggering paths? understanding and characterizing learning-based vulnerability detectors, *IEEE Trans. Dependable Secur. Comput.* (2022).
- [8] I. Sutskever, O. Vinyals, Q.V. Le, Sequence to sequence learning with neural networks, in: *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014*, pp. 3104–3112.
- [9] M. Schuster, K.K. Paliwal, Bidirectional recurrent neural networks, *IEEE Trans. Signal Process.* 45 (11) (1997) 2673–2681.
- [10] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: *25th Annual Network and Distributed System Security Symposium, NDSS 2018*.
- [11] T. Wu, L. Chen, G. Du, C. Zhu, G. Shi, Self-attention based automated vulnerability detection with effective data representation, in: *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking, ISPA/BDCloud/SocialCom/SustainCom, 2021*, pp. 892–899.
- [12] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, *IEEE Trans. Neural Netw.* 20 (1) (2008) 61–80.
- [13] Y. Li, D. Tarlow, M. Brockschmidt, R.S. Zemel, Gated graph sequence neural networks, in: *4th International Conference on Learning Representations, ICLR 2016*.
- [14] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, 2014, arXiv preprint arXiv:1412.3555.
- [15] Y. Zhou, S. Liu, J.K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019*, pp. 10197–10207.
- [16] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, Deepwukong: Statically detecting software vulnerabilities using deep graph neural network, *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 30 (3) (2021) 1–33.
- [17] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: *2014 IEEE Symposium on Security and Privacy, SP 2014*, pp. 590–604.
- [18] I. Neamtiu, J.S. Foster, M.W. Hicks, Understanding source code evolution using abstract syntax tree matching, in: *Proceedings of the 2005 International Workshop on Mining Software Repositories, MSR 2005*.
- [19] W. Guo, Y. Fang, C. Huang, H. Ou, C. Lin, Y. Guo, HyVulDect: a hybrid semantic vulnerability mining system based on graph neural network, *Comput. Secur.* 121 (2022) 102823.
- [20] C. Ni, X. Guo, Y. Zhu, X. Xu, X. Yang, Function-level vulnerability detection through fusing multi-modal knowledge, in: *2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2023*, pp. 1911–1918.
- [21] W. Guo, Y. Fang, C. Huang, H. Ou, C. Lin, Y. Guo, HyVulDect: a hybrid semantic vulnerability mining system based on graph neural network, *Comput. Secur.* 121 (2022) 102823.
- [22] W. Tao, X. Su, J. Wan, H. Wei, W. Zheng, Vulnerability detection through cross-modal feature enhancement and fusion, *Comput. Secur.* 132 (2023) 103341.
- [23] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, X. Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020*, pp. 3438–3445.
- [24] W. Cai, J. Chen, J. Yu, L. Gao, A software vulnerability detection method based on deep learning with complex network analysis and subgraph partition, *Inf. Softw. Technol.* 164 (2023) 107328.
- [25] W. Kim, B. Son, I. Kim, Vilt: Vision-and-language transformer without convolution or region supervision, in: *International Conference on Machine Learning, PMLR, 2021*, pp. 5583–5594.
- [26] H. Hanif, S. Maffei, Vulberta: Simplified source code pre-training for vulnerability detection, in: *2022 International Joint Conference on Neural Networks, IJCNN, IEEE, 2022*, pp. 1–8.
- [27] M. Fu, C. Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: *Proceedings of the 19th International Conference on Mining Software Repositories, 2022*, pp. 608–620.
- [28] X. Meng, S. Lu, X. Wang, X. Liu, C. Hu, Improving vulnerability detection with hybrid code graph representation, in: *2023 30th Asia-Pacific Software Engineering Conference, APSEC, IEEE, 2023*, pp. 259–268.
- [29] X.-C. Wen, Y. Chen, C. Gao, H. Zhang, J.M. Zhang, Q. Liao, Vulnerability detection with graph simplification and enhanced graph representation learning, in: *2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, IEEE, 2023*, pp. 2275–2286.
- [30] Z. Liu, H. Hu, Y. Lin, Z. Yao, Z. Xie, Y. Wei, J. Ning, Y. Cao, Z. Zhang, L. Dong, et al., Swin transformer v2: Scaling up capacity and resolution, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2022*, pp. 12009–12019.
- [31] M.S. Schlichtkrull, T.N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *The Semantic Web - 15th International Conference, Vol. 10843, 2018*, pp. 593–607.
- [32] J. Ferrante, K.J. Ottenstein, J.D. Warren, The program dependence graph and its use in optimization, *ACM Trans. Program. Lang. Syst.* 9 (3) (1987) 319–349.
- [33] D. Bruschi, L. Martignoni, M. Monga, Detecting self-mutating malware using control-flow graph matching, in: *Detection of Intrusions and Malware & Vulnerability Assessment, Third International Conference, Vol. 4064, DIMVA 2006*, pp. 129–143.
- [34] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, in: *1st International Conference on Learning Representations, ICLR 2013*.
- [35] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: Transformers for image recognition at scale, 2020, arXiv preprint arXiv:2010.11929.
- [36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [37] S. Gold, A. Rangarajan, et al., Softmax to softsign: Neural network algorithms for combinatorial optimization, *J. Artif. Neural Netw.* 2 (4) (1996) 381–399.
- [38] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *3rd International Conference on Learning Representations, ICLR 2015*.

- [39] P.-T. De Boer, D.P. Kroese, S. Mannor, R.Y. Rubinstein, A tutorial on the cross-entropy method, *Ann. Oper. Res.* 134 (2005) 19–67.
- [40] X. Glorot, A. Bordes, Y. Bengio, Deep sparse rectifier neural networks, in: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, Vol. 15, AISTATS 2011, pp. 315–323.
- [41] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, H. Jin, VulCNN: An image-inspired scalable vulnerability detection system, in: *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022*, pp. 2365–2376.
- [42] Y. Kim, Convolutional neural networks for sentence classification, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014*, pp. 1746–1751.